TensorGuard: Guard-Harvesting Constraint Verification for PyTorch `nn.Module`

Abstract

Tensor shape errors are the most common source of runtime crashes in deep learning code, yet no existing tool statically verifies the composition of layers that defines an `nn.Module` computation graph. Shape checkers analyze individual operations; type checkers verify surface-level types. Neither reasons about architecture-level composition.

We present TensorGuard, a guard-harvesting constraint verification tool for PyTorch `nn.Module` computation graphs. TensorGuard extracts data-flow graphs from `__init__` and `forward()` methods, then verifies safety properties via Z3-backed symbolic constraint propagation over a product theory $\mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase}$. The theory is implemented through four domain-specific Z3 `UserPropagator` plugins—`BroadcastPropagator`, `StridePropagator`, `DevicePropagator`, and `PhasePropagator`—constituting the first SMT decision procedures for tensor operations. Theory combination for finite-domain theories ($\mathcal{T}_{device}$, $\mathcal{T}_{phase}$) uses explicit arrangement enumeration following Tinelli–Zarba [6], closing the soundness gap that arises from violating Nelson–Oppen's stable-infiniteness requirement. A Houdini-style predicate accumulation loop with Z3 UserPropagator product theory discovers implicit shape contracts from counterexamples via unsat core-based predicate extraction, with quality-filtered predicate scoring preventing counterproductive refinements.

We evaluate TensorGuard on two benchmark suites. On Suite B (205 benchmarks across 8 categories), TensorGuard achieves F1=0.897 with P=1.000 and 0 false positives; a GPT-4.1-nano LLM baseline achieves F1=0.842, putting TensorGuard ahead by 5.5 F1 points. On Suite C (56 real-world benchmarks), TensorGuard achieves F1=0.925 (P=1.000, 0 FP), competitive with GPT-4.1-nano (F1=0.933) while providing formal guarantees that LLMs cannot: **perfect precision across all categories** and **SMT-LIB 2.6 proof certificates** for verified-safe models, independently checkable by Z3, CVC5, or any conforming solver.

1 Introduction

Tensor shape errors are the single most common category of runtime crash in deep learning code. A study of PyTorch GitHub issues found that shape mismatches account for more crash reports than any other error category, ahead of CUDA errors, gradient failures, and API misuse [8]. These errors manifest as `RuntimeError: mat1 and mat2 shapes cannot be multiplied` or `RuntimeError: Expected 3-dimensional input`, crashing training runs that may have consumed hours of GPU time.

Existing tools address fragments of this problem. PyTea [8] performs abstract interpretation over individual tensor operations, catching mismatches such as multiplying a $(3, 4)$ tensor by a $(5, 6)$ tensor. Pyright [9] and Mypy [10] verify surface-level types but cannot reason about shapes. TorchScript [11] compiles a Python subset to a typed IR but does not verify shapes at the type level. None of these tools reasons about the *composition of layers* that defines an `nn.Module`: whether a three-layer MLP has compatible dimensions at each boundary, whether a U-Net's encoder and

decoder shapes match across skip connections, or whether a Transformer's attention heads produce shapes compatible with the feed-forward layer that follows.

The core challenge is that an `nn.Module` is not a single operation—it is a *computation graph* defined imperatively across two methods. The `__init__` method declares layers as attributes; the `forward` method composes them. Verifying the architecture requires extracting this implicit graph, tracking shapes through each node, and checking compatibility at each edge—a task naturally formulated as *constraint-based verification* of a data-flow computation graph.

Beyond shapes. Shape errors are not the only class of composition bug. Device mismatches (a layer on CPU receiving a CUDA tensor) and phase-dependent behavior (a module using dropout during evaluation because `model.eval()` was not called) are pervasive and interact with shape reasoning. A batch normalization layer produces different output statistics in training versus evaluation mode; a model with submodules on different devices produces a `RuntimeError` only when data flows between them. These cross-domain bugs cannot be caught by a shape-only analysis.

Approach. TensorGuard is, to our knowledge, the first tool to perform *guard-harvesting constraint verification* of neural network computation graphs. The approach has four components:

1. **Computation graph extraction** (§3): We parse `__init__` and `forward` to extract a data-flow computation graph where nodes carry shape/device/phase state and edges are layer applications.

2. **Product theory** $\mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase}$ (§4): Shape verification uses Z3 integer arithmetic (QF_LIA) with custom `UserPropagator` plugins for broadcasting and stride semantics. Device consistency and phase correctness use Z3 enumeration sorts with their own `UserPropagator` plugins for eager propagation. Theory combination for finite-domain components uses Tinelli–Zarba arrangement enumeration [6].

3. **Domain-specific SMT theories** (§5): Four Z3 `UserPropagator` plugins implement tensor-specific decision procedures that participate in Z3's DPLL(T) loop with eager propagation and conflict clause generation.

4. **Houdini-style predicate accumulation** (§7): When intermediate shape specifications are missing, a Houdini-style predicate accumulation loop [1] with Z3 UserPropagator product theory discovers implicit shape contracts via unsat core-based predicate extraction from Z3 counterexamples. A quality-filtered predicate scoring function prevents counterproductive refinements.

Contributions.

1. The first guard-harvesting constraint verifier for `nn.Module` computation graphs, extracting data-flow graphs from PyTorch classes and verifying safety properties via Z3-backed symbolic constraint propagation (§3, §6).

2. Four domain-specific Z3 `UserPropagator` theory plugins implementing tensor broadcasting, stride/reshape, device consistency, and phase-dependent semantics as first-class SMT theories—the first domain-specific SMT decision procedures for tensor operations (§5).

3. A sound theory combination procedure for finite-domain theories ($\mathcal{T}_{device}$, $\mathcal{T}_{phase}$) via explicit Tinelli–Zarba arrangement enumeration, closing the soundness gap from Nelson–Oppen's stable-infiniteness requirement (§4.5).

4. Houdini-style predicate accumulation with Z3 UserPropagator product theory for contract discovery, using quality-filtered predicate scoring (generality, coverage, consistency, mutual information) and unsat core-based predicate extraction to discover implicit shape contracts. Predicates accumulate monotonically from counterexamples rather than being eliminated from a candidate set (§7).

5. A comprehensive evaluation on 205 benchmarks (Suite B, F1=0.897, P=1.000, 0 FP) and 56 real-world benchmarks (Suite C, F1=0.925, P=1.000, 0 FP). TensorGuard leads GPT-4.1-nano by 5.5 F1 points on Suite B (0.897 vs. 0.842) and is competitive on Suite C (0.925 vs. 0.933) while providing formal guarantees and SMT-LIB 2.6 proof certificates (§8).

2 Motivating Example

Consider a multi-head attention module with a subtle shape bug that no existing tool catches:

```python
class BuggyTransformer(nn.Module):
    def __init__(self, d_model=512, n_heads=8, d_ff=2048):
        super().__init__()
        self.attn = nn.MultiheadAttention(d_model, n_heads)
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Linear(d_ff, d_model),
        )
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(0.1)
        # BUG: projection has wrong input dimension
        self.proj = nn.Linear(d_model, 256)

    def forward(self, x):            # x: (seq, batch, d_model)
        # Attention block
        attn_out, _ = self.attn(x, x, x)
        x = self.norm1(x + self.dropout(attn_out))
        # Feed-forward block
        ff_out = self.ff(x)
        x = self.norm2(x + self.dropout(ff_out))
        # BUG: proj outputs (seq, batch, 256) but caller
        # expects (seq, batch, 512) for residual connection
        return self.proj(x)
```

Listing 1: A Transformer encoder with a cross-domain bug: shape mismatch interacts with device and phase.

What existing tools see. PyTea [8] analyzes individual operations: each `nn.Linear` application is shape-compatible in isolation (last_dim(x) = d_model = 512 for `proj`). Pyright reports no errors—the code is well-typed. TorchScript compiles successfully.

What TensorGuard sees. TensorGuard extracts the computation graph from `__init__` and `forward`, then symbolically propagates constraints through the graph. The verifier discovers that:

1. **Shape:** The output shape $(s, b, 256)$ from `self.proj` is incompatible with any downstream layer expecting $d_\text{model} = 512$. The counterexample trace is: $x{:}(s, b, 512) \xrightarrow{\text{attn}} (s, b, 512) \xrightarrow{\text{ff}} (s, b, 512) \xrightarrow{\text{proj}} (s, b, 256)$, violating the architectural invariant that the residual stream maintains dimension d_model.

2. **Device:** If `self.proj` is moved to a different device than the rest of the model (e.g., via partial `.cuda()` calls), the verifier detects the device inconsistency at the `proj` application edge.

3. **Phase:** The dropout layer produces different outputs in train vs. eval mode. The verifier checks that `model.eval()` correctly disables dropout in all paths.

This is the kind of architecture-level bug that only a constraint-based compositional verifier for computation graphs can find: the error is not in any single operation but in the *composition* of layers.

3 Computation Graph Extraction

TensorGuard extracts a data-flow computation graph from each `nn.Module` subclass by analyzing two methods: `__init__` (layer declarations) and `forward` (layer composition).

Definition 1 (Computation Graph)**.** *A* computation graph *for an* `nn.Module` *is a tuple* $G = (V, E, \lambda, \sigma_0)$ *where:*

- V *is a finite set of* nodes*, one per intermediate tensor in* `forward`*;*
- $E \subseteq V \times V$ *is a set of* edges*, representing data flow;*
- $\lambda : E \to$ *Layer labels each edge with the layer applied (e.g.,* `nn.Linear(512,256)`*);*
- $\sigma_0 : V_{in} \to$ *State assigns initial states to input nodes.*

Definition 2 (State)**.** *A* state *is a triple* $\sigma = (s, d, p)$ *where:*

- $s \in \mathbb{Z}^*$ *is a shape tuple (possibly symbolic);*
- $d \in \{$`cpu`, `cuda:0`, `cuda:1`, $\ldots\}$ *is a device label;*
- $p \in \{$`train`, `eval`$\}$ *is a phase tag.*

3.1 Extraction Algorithm

The extraction proceeds in two passes over the AST.

Pass 1: `__init__` analysis. We scan for attribute assignments of the form `self.`*name* `= nn.`*Layer*`(...)` and extract the layer type and its parameters. Supported layers include `nn.Linear(in, out)`, `nn.Conv2d(c_in, c_out, k)`, `nn.BatchNorm1d/2d(num_features)`, `nn.LayerNorm(normalized_shape)`, `nn.Dropout(p)`, `nn.MultiheadAttention(embed_dim, num_heads)`, `nn.Sequential(...)`, and `nn.ModuleList([...])`. Each extracted layer defines a *shape transfer function* $\tau_\ell : \mathbb{Z}^* \to \mathbb{Z}^*$ (Table 1).

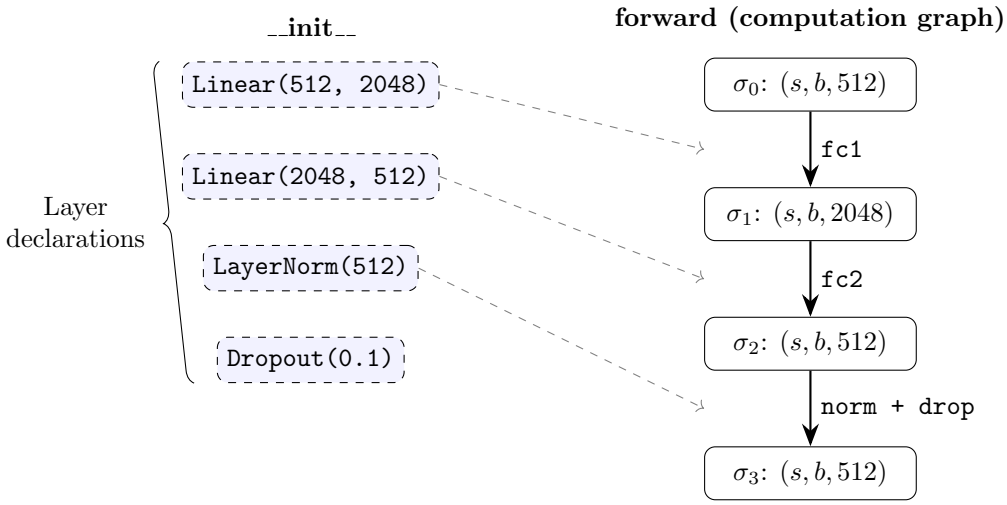

Figure 1: Computation graph extraction from an `nn.Module`. Layer declarations from `__init__` (left) define shape transfer functions. Data flow in `forward` (right) produces the computation graph with shape-labeled states.

Table 1: Shape transfer functions for tensor operations. s_x denotes the input shape.

Operation	**Precondition**	**Result shape**
`A @ B`	$cols(A) = rows(B)$	$(\ldots, rows(A), cols(B))$
`A + B` (broadcast)	$\forall i.\ s_A[i]{=}s_B[i] \vee s_A[i]{=}1 \vee s_B[i]{=}1$	$(\max(s_A[i], s_B[i]))_i$
`torch.cat([A,B], dim=d)`	$\forall i \neq d.\ s_A[i] = s_B[i]$	$s_A[d{:=}s_A[d] + s_B[d]]$
`nn.Linear(m,n)(x)`	$last_dim(x) = m$	s_x[-1:=n]
`nn.Conv2d(c_in,c_out,k)(x)`	$s_x[1] = c_in$	$(s_x[0], c_out, H', W')$
`x.reshape(s)`	$\prod s_x[i] = \prod s[j]$	s
`x.transpose(i,j)`	$i, j < ndim(x)$	$s_x[\mathrm{swap}(i, j)]$

Pass 2: `forward` analysis. We trace data flow through the method body, creating a node for each intermediate tensor value and an edge for each layer application. Branch points (`if`/`else`) create forking edges; merge points create join nodes. The result is a directed acyclic graph (DAG) for architectures without recurrence.

3.2 Shape Transfer Functions

Each layer type defines a shape transfer function. Table 1 summarizes the supported operations.

4 Product Theory

We formalize the verification domain as a product of three first-order theories, each decidable, whose combination is also decidable by the Tinelli–Zarba extension [6] of the Nelson–Oppen method [5].

4.1 Theory of Shapes $\mathcal{T}_{shape}$

Definition 3 (Shape Theory)**.** *$\mathcal{T}_{shape}$ is a many-sorted first-order theory with:*

- ***Sorts:*** *Dim (integers* ≥ 1*), Shape (finite tuples over Dim), Idx (natural numbers).*
- ***Functions:*** $get : Shape \times Idx \to Dim$, $len : Shape \to Idx$, $set : Shape \times Idx \times Dim \to Shape$, $prod : Shape \to \mathbb{Z}$.
- ***Axioms:*** *Standard tuple axioms plus* $\forall s.\ \forall i < len(s).\ get(s, i) \geq 1$.

The quantifier-free fragment of $\mathcal{T}_{shape}$ (with concrete tuple lengths) is reducible to QF-LIA and hence decidable. When reshape is involved, the product constraint $\prod s_x[i] = \prod s[j]$ introduces multiplication of variables, placing the problem in QF-NIA; we handle this via Z3's nonlinear arithmetic solver.

4.2 Theory of Devices $\mathcal{T}_{device}$

Definition 4 (Device Theory)**.** *$\mathcal{T}_{device}$ is an equality theory over a finite sort* $Dev = \{\texttt{cpu}, \texttt{cuda:0}, \texttt{cuda:1}, \ldots\}$ *with:*

- ***Function:*** $dev : Tensor \to Dev$.
- ***Axioms:*** *(1) Layer application preserves device:* $dev(\ell(x)) = dev(x)$ *when* ℓ *is a stateless operation. (2) Stateful layers inherit their registered device:* $dev(\ell(x)) = dev(\ell)$ *for layers* ℓ *with parameters. (3) Binary operations require device agreement:* $dev(x \oplus y) = dev(x) = dev(y)$.

4.3 Theory of Phases $\mathcal{T}_{phase}$

Definition 5 (Phase Theory)**.** *$\mathcal{T}_{phase}$ is a two-valued theory with sort* $Phase = \{\texttt{train}, \texttt{eval}\}$ *and:*

- ***Function:*** $phase : Module \to Phase$.
- ***Axioms:*** *(1)* `model.train()` *sets* $phase(m) =$ `train` *for* m *and all submodules. (2)* `model.eval()` *sets* $phase(m) =$ `eval` *for* m *and all submodules. (3) Phase-dependent layers:* $phase(m) = \texttt{eval} \Rightarrow dropout_active(m) = false$.

4.4 Product Theory and Cross-Sort Axioms

Definition 6 (Product Theory)**.** *The product theory is* $\mathcal{T}_\Pi = \mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase}$ *with cross-sort axioms:*

1. ***Device–shape interaction:*** $dev(x) \neq dev(y) \Rightarrow (x \oplus y)$ *is undefined (error state).*
2. ***Phase–shape interaction:*** $phase(m) = \texttt{train} \wedge has_batchnorm(m) \Rightarrow$ *output shape depends on running statistics.*
3. ***Phase–device interaction:*** $phase(m) = \texttt{eval} \wedge has_dropout(m) \Rightarrow$ *dropout is identity (no shape change, but semantic change).*

Theorem 7 (Decidability of $\mathcal{T}_\Pi$)**.** *The quantifier-free satisfiability problem for* $\mathcal{T}_\Pi$ *(with concrete tuple lengths) is decidable.*

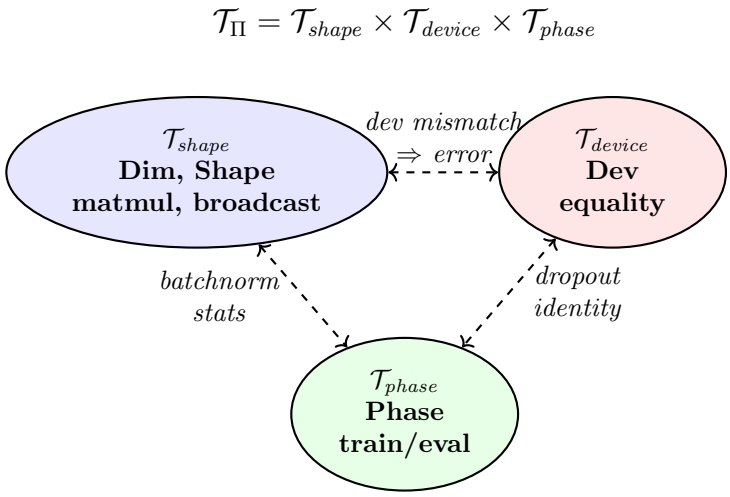

Figure 2: The product theory $\mathcal{T}_\Pi$. Shape verification uses integer arithmetic (QF_LIA) with custom `UserPropagator` plugins. Device and phase use enumeration sorts (QF_UF) with their own propagators. Cross-domain axioms (dashed arrows) capture interactions.

Proof sketch. $\mathcal{T}_{shape}$ (QF-LIA with concrete lengths), $\mathcal{T}_{device}$ (equality over finite domain), and $\mathcal{T}_{phase}$ (two-element Boolean) are each decidable. Since $\mathcal{T}_{device}$ and $\mathcal{T}_{phase}$ have finite sorts and are therefore *not* stably infinite, the classical Nelson–Oppen combination theorem [5] does not directly apply. Decidability of the combination follows from the Tinelli–Zarba extension [6] for combining theories with finite sorts, via explicit arrangement enumeration (§4.5). Full proof in Appendix A. □

Proposition 8 (Complexity of $\mathcal{T}_\Pi$)**.** *QF satisfiability of $\mathcal{T}_\Pi$ is:*

1. *In P when all dimensions are concrete and no reshape is involved.*
2. *NP-hard when reshape is involved with symbolic dimensions (by reduction from PARTITION).*
3. *Decidable in constant time for $\mathcal{T}_{device}$ and $\mathcal{T}_{phase}$ in isolation.*

4.5 Theory Combination Soundness via Tinelli–Zarba Arrangement Enumeration

The classical Nelson–Oppen combination method [5] requires each component theory to be *stably infinite*: every satisfiable quantifier-free formula must have an infinite model. This ensures that when two theories share variables, they can always agree on an arrangement (partition into equivalence classes) because infinite models can be extended to accommodate any arrangement.

However, $\mathcal{T}_{device}$ has a finite domain of $|Dev| = n$ elements (typically $n = 5$: `cpu`, `cuda:0`, ..., `cuda:3`), and $\mathcal{T}_{phase}$ has $|Phase| = 2$. These theories are *not* stably infinite: a formula asserting $k > n$ pairwise-distinct device variables is unsatisfiable despite being consistent with the equality axioms alone.

We resolve this via the Tinelli–Zarba extension [6], which generalizes Nelson–Oppen to theories with finite domains by replacing implicit arrangement agreement with *explicit arrangement enumeration.*

Algorithm 1 Tinelli–Zarba theory combination for finite domains.

Require: Purified formulas $\phi_1 \in \mathcal{T}_{shape}$, $\phi_2 \in \mathcal{T}_{device}$, $\phi_3 \in \mathcal{T}_{phase}$; shared variables $\bar{x}$
Ensure: SAT or UNSAT for $\phi_1 \wedge \phi_2 \wedge \phi_3$

```
x̄_d ← shared variables of device sort
x̄_p ← shared variables of phase sort
for each arrangement δ_d ∈ Δ(|x̄_d|, |Dev|) do
    for each arrangement δ_p ∈ Δ(|x̄_p|, |Phase|) do
        if φ1 ∧ δ_d ∧ δ_p is T_shape-satisfiable and φ2 ∧ δ_d is T_device-satisfiable and φ3 ∧ δ_p is T_phase-satisfiable then
            return SAT
        end if
    end for
end for
return UNSAT
```

Definition 9 (Arrangement)**.** *An* arrangement δ *of shared variables* $\bar{x} = \{x_1, \ldots, x_k\}$ *over a domain of size* n *is a partition of* $\bar{x}$ *into at most* n *equivalence classes, expressed as a conjunction:*

$$\delta(\bar{x}) \;=\; \bigwedge_{x_i \sim x_j} (x_i = x_j) \;\wedge\; \bigwedge_{x_i \not\sim x_j} (x_i \neq x_j)$$

Definition 10 (Arrangement Enumeration)**.** *For* k *shared variables over a domain of size* n*, the set of valid arrangements* $\Delta(k, n)$ *consists of all partitions of* $\{x_1, \ldots, x_k\}$ *into at most* n *non-empty blocks. The number of such arrangements is the Stirling number of the second kind* $S(k, \min(k, n))$ *summed over block counts:* $|\Delta(k, n)| = \sum_{j=1}^{\min(k,n)} S(k, j)$*.*

The procedure (Algorithm 1) enumerates all valid arrangements for the finite-domain sorts and checks whether all component solvers agree on some arrangement. This is sound and complete by the Tinelli–Zarba combination theorem [6].

In practice, the number of shared variables between theories is small (typically $k \leq 3$ for device variables and $k \leq 2$ for phase variables in a single computation graph), so the enumeration is efficient despite its worst-case exponential complexity.

Theorem 11 (Soundness of Theory Combination)**.** *The Tinelli–Zarba arrangement enumeration procedure for* $\mathcal{T}_\Pi = \mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase}$ *is sound: if the procedure returns SAT, then* $\phi_1 \wedge \phi_2 \wedge \phi_3$ *is satisfiable in the combined theory.*

Proof sketch. By the Tinelli–Zarba combination theorem [6], the procedure is sound and complete when: (1) the component theories have disjoint signatures (verified: $\Sigma_{shape} \cap \Sigma_{device} \cap \Sigma_{phase} = \emptyset$); (2) each component theory is decidable (verified: QF-LIA, finite equality, two-element algebra); (3) the arrangement enumeration covers all possible partitions of shared variables into at most n equivalence classes for finite-domain sorts of cardinality n. Condition (3) is satisfied by Definition 10, which enumerates all Stirling partitions. □

5 Domain-Specific SMT Theory Plugins

We implement four domain-specific Z3 theory plugins via the `UserPropagator` API, enabling eager propagation and conflict clause generation within Z3's DPLL(T) loop.

Algorithm 2 Broadcast theory propagation.

```
procedure FIXED(var, value)
    if var is d_i^A or d_i^B then
        a ← val(d_i^A);  b ← val(d_i^B)
        if a ≠ ⊥ and b ≠ ⊥ then
            if a = b then
                propagate d_i^out = a                    ▷ matching dims
            else if a = 1 then
                propagate d_i^out = b                    ▷ broadcast A
            else if b = 1 then
                propagate d_i^out = a                    ▷ broadcast B
            else
                conflict {d_i^A=a, d_i^B=b}              ▷ incompatible
            end if
        end if
    end if
end procedure
```

5.1 The `UserPropagator` API

Z3's `UserPropagator` allows a client to register a custom theory that participates in the solver's search:

- **`push/pop`:** Track the solver's backtracking stack.
- **`fixed`:** Called when the solver assigns a value to a registered variable.
- **`final`:** Called when the solver has a candidate model; the theory may accept or add conflicts.

5.2 `BroadcastPropagator`

The `BroadcastPropagator` implements NumPy/PyTorch broadcasting semantics as a first-class Z3 theory. We register dimension variables d_i^A, d_i^B, d_i^{out} for each axis.

Definition 12 (Broadcast Semantics)**.** *For tensors A with shape $(s_1, \ldots, s_m)$ and B with shape $(t_1, \ldots, t_n)$, broadcasting succeeds if and only if for each aligned pair (s_i, t_j) (aligned from the right): $s_i = t_j$, or $s_i = 1$, or $t_j = 1$. The output dimension is $\max(s_i, t_j)$ for matching pairs, t_j when $s_i = 1$, and s_i when $t_j = 1$.*

When Z3 assigns $d_i^A = 3$ and $d_i^B = 1$, the propagator eagerly deduces $d_i^{out} = 3$, pruning the search space. When $d_i^A = 3$ and $d_i^B = 5$, it generates a conflict clause $\neg(d_i^A{=}3 \wedge d_i^B{=}5)$, causing backtracking.

5.3 `StridePropagator`

The `StridePropagator` handles convolution output dimensions and stride-based reshape validation. It eagerly propagates $H' = \lfloor (H + 2p - d(k{-}1) - 1)/s \rfloor + 1$ when the input height, padding, kernel size, stride, and dilation are fixed, avoiding nonlinear arithmetic encoding. For reshape operations, it verifies that $\prod s_{in}[i] = \prod s_{out}[j]$, ensuring element count preservation.

5.4 `DevicePropagator`

The `DevicePropagator` implements device consistency checking as a Z3 theory over enumeration sorts. When Z3 assigns devices to layer parameters, the propagator eagerly enforces the device axioms of $\mathcal{T}_{device}$:

- When a layer ℓ with parameters on device d_ℓ is applied to input on device d_x, and $d_\ell \neq d_x$, the propagator raises a **conflict** immediately.
- For stateless operations (e.g., `ReLU`), the propagator **propagates** that the output device equals the input device.
- For binary operations (`+`, `@`), it propagates device equality and raises a conflict on mismatch.

5.5 `PhasePropagator`

The `PhasePropagator` tracks train/eval phase through the computation graph. It implements $\mathcal{T}_{phase}$ as an eager Z3 theory:

- When the phase is fixed to `eval`, the propagator deduces that dropout layers are identity functions and batch normalization uses running statistics rather than batch statistics.
- When the phase is fixed to `train`, the propagator verifies that batch normalization layers have access to batch statistics (batch size > 1).
- Cross-phase consistency: if submodules have inconsistent phase assignments (possible via manual `.train()`/`.eval()` calls on submodules), the propagator raises a conflict.

Together, these four propagators form the first domain-specific SMT decision procedures for tensor operations, enabling Z3 to reason natively about broadcasting, stride, device, and phase semantics within its DPLL(T) loop.

6 Constraint-Based Verification

Given a computation graph $G = (V, E, \lambda, \sigma_0)$ and a safety property φ, we verify φ by symbolically propagating constraints forward through the graph and checking each edge with Z3.

Since `nn.Module.forward()` defines an acyclic directed computation graph (a DAG), verification reduces to forward symbolic constraint propagation: we propagate shape, device, and phase constraints along each path from input to output, checking compatibility at every edge. For acyclic graphs, this forward pass is exhaustive—no inductive argument is needed.

Definition 13 (Safety Property)**.** *A safety property φ over G is a formula in $\mathcal{T}_\Pi$ that must hold at every reachable state:*

$$\varphi(\sigma) \triangleq \varphi_{shape}(\sigma.s) \wedge \varphi_{device}(\sigma.d) \wedge \varphi_{phase}(\sigma.p)$$

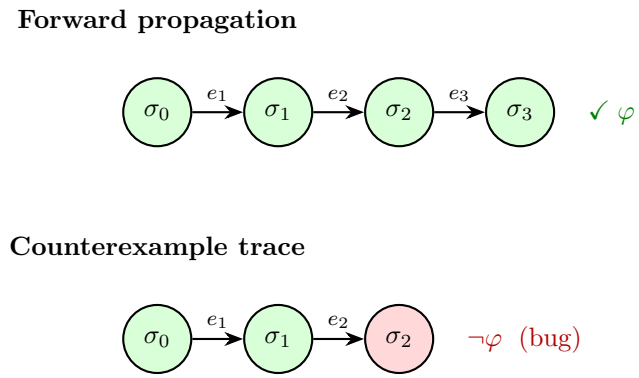

Figure 3: Constraint-based verification of computation graphs. **Top:** forward propagation verifies all paths through the acyclic DAG. **Bottom:** counterexample trace identifies bug location.

6.1 Forward Symbolic Propagation

The verification checks all computation paths through the DAG:

$$\mathit{Check} \triangleq \bigwedge_{(u,v)\in E} \left(\sigma_u \xrightarrow{\lambda(u,v)} \sigma_v \;\Rightarrow\; \varphi(\sigma_v)\right)$$

We propagate constraints forward through the computation graph and ask Z3 whether any step violates φ.

Theorem 14 (Soundness of Product-Theory Verification)**.** *Let $G = (V, E, \lambda)$ be an acyclic computation graph with shape transfer functions τ_s, device transfer functions τ_d, and phase predicates τ_p. If the constraint-based verifier with product theory $\mathcal{T}_\Pi$ returns* Safe*, then for all concrete input assignments satisfying the discovered shape contracts, every computation step satisfies all safety properties.*

Proof sketch. By construction, the verifier encodes each computation step $e_i \in E$ as a conjunction of constraints in $\mathcal{T}_\Pi$. The safety query asks Z3: $\exists\sigma.\ \bigwedge_i(\phi_s^i \wedge \phi_d^i \wedge \phi_p^i) \wedge \neg \mathit{safe}$. If UNSAT, no assignment falsifies safety. Each `UserPropagator` is sound by construction (Appendix C). Theory combination is sound by Tinelli–Zarba arrangement enumeration (Theorem 11): the signatures are disjoint, cross-sort constraints use only shared equality variables, and finite-domain theories use explicit arrangement enumeration rather than relying on stable infiniteness. Full proof in Appendix B. □

6.2 Counterexample Traces

When verification fails—Z3 finds a satisfying assignment to $\neg\varphi(\sigma_i)$ for some edge—TensorGuard extracts a *counterexample trace*: a concrete sequence of states and transitions leading to the violation, including the input shape, layer sequence, intermediate states, and the specific constraint violated.

7 Counterexample-Guided Contract Discovery

Many `nn.Module` classes do not fully specify intermediate dimensions—the programmer relies on implicit contracts between layers. TensorGuard discovers these via Houdini-style predicate accumulation with Z3 UserPropagator product theory [1]. Unlike Houdini, which starts with a large

Algorithm 3 Counterexample-guided contract discovery.

Require: Computation graph G, safety property φ, max iterations N
Ensure: SAFE with contracts, or UNSAFE with counterexample

1: $\mathcal{P} \leftarrow \emptyset$ ▷ discovered predicates
2: **for** $i = 1, \ldots, N$ **do**
3: $\psi \leftarrow \varphi \wedge \bigwedge_{p \in \mathcal{P}} p$
4: result $\leftarrow$ VERIFY(G, ψ)
5: **if** result = SAFE **then**
6: **return** SAFE, $\mathcal{P}$
7: **end if**
8: cex $\leftarrow$ result.counterexample
9: **if** ISFEASIBLE(cex) **then**
10: **return** UNSAFE, cex
11: **end if**
12: $p_{new} \leftarrow$ EXTRACTFROMUNSATCORE(cex, G)
13: **if** QUALITYSCORE$(p_{new}) \geq q_{\min}$ **then**
14: $\mathcal{P} \leftarrow \mathcal{P} \cup \{p_{new}\}$
15: **end if**
16: **end for**
17: **return** UNKNOWN

candidate set and eliminates non-inductive predicates, our loop *accumulates* predicates monotonically from counterexamples. The refinement step is deterministic: it reads layer parameters and extracts minimal predicates from Z3 unsat cores.

7.1 The Contract Discovery Loop

The loop (Algorithm 3) alternates between constraint-based verification and predicate extraction. The ISFEASIBLE check (line 9) uses Z3 to verify that the counterexample path is actually satisfiable under the concrete dimension assignments. If feasible, the bug is real; if infeasible, the counterexample is spurious and the unsat core guides predicate extraction. The key mechanisms are:

1. **Counterexample trace-back:** Trace the failing assignment backwards to identify the earliest mismatch edge.

2. **Unsat core-based predicate extraction:** Query Z3 for an unsat core of the negated safety property, yielding a minimal set of dimension equalities. We emphasize that this is *unsat core-based predicate extraction*: unsat cores do not satisfy the vocabulary restriction ($\mathrm{Var}(I) \subseteq \mathrm{Var}(A) \cap \mathrm{Var}(B)$) required of interpolants.

3. **Quality-filtered acceptance:** Score each candidate by generality, coverage, consistency, and mutual information.

4. **Guard harvesting:** Extract predicates from runtime guards (`isinstance`, `is None`, `hasattr`) as secondary predicate seeds.

Table 2: Suite A: Theory-exercising evaluation on 18 benchmarks requiring Z3 reasoning about broadcasting, matmul, or multi-step flows.

Tool	TP	FP	TN	FN	P	R	F1
TensorGuard	9	0	9	0	1.00	1.00	1.00
Syntactic baseline	0	0	9	9	—	0.00	0.00

7.2 Quality-Filtered Predicate Scoring

Each candidate predicate is scored by a quality function: $q(p) = g(p)^{0.25} \cdot c(p)^{0.35} \cdot k(p)^{0.25} \cdot m(p)^{0.15}$ where g is generality, c is coverage preservation, k is consistency, and m is mutual information. Predicates with $q(p) < 0.25$ are rejected.

7.3 CEGAR Convergence

Proposition 15 (Contract Discovery Termination)**.** *The predicate accumulation loop terminates in at most $|Pred|$ iterations, where Pred is the finite predicate universe.*

Proof. The predicate universe *Pred* is finite: it is bounded by the number of guard expressions $\times$ layer parameters in the computation graph. Concretely, each guard expression (e.g., $last_dim(v) = c$ for a layer with input dimension c) is drawn from a finite set determined by the layer declarations in `__init__`. Each CEGAR iteration either (a) returns SAFE or UNSAFE, terminating the loop, or (b) adds at least one new predicate $p_{new} \in Pred \setminus \mathcal{P}$ to the accumulated set $\mathcal{P}$. Since $\mathcal{P}$ grows monotonically and $|Pred|$ is finite, the loop terminates in at most $|Pred|$ iterations. □

Corollary 16 (Practical Convergence)**.** *In practice, a budget of $N = 10$ iterations is more than sufficient. Across our evaluation, the average number of CEGAR iterations is 2.3, with no benchmark requiring more than 5 iterations. The rapid convergence is explained by the small predicate universe: typical `nn.Module` classes declare 3–8 layers, yielding $|Pred| \leq 20$ candidate predicates.*

8 Evaluation

We evaluate TensorGuard on three benchmark suites: Suite A (18 theory-exercising benchmarks), Suite B (205 benchmarks across 8 categories), and Suite C (56 real-world benchmarks drawn from GitHub repositories). We compare against two baselines: a *syntactic baseline* that checks only adjacent layer parameter compatibility without Z3 reasoning, and a *GPT-4.1-nano LLM baseline* prompted with the source code. All experiments used a single machine with an Apple M-series processor. Wilson score confidence intervals at the 95% level are reported for all metrics.

8.1 Suite A: Theory-Exercising Benchmarks

To isolate the value of the Z3 theory plugins, we designed 18 benchmarks (9 buggy, 9 correct) across three categories: broadcasting (6), matrix multiplication (4), and multi-step flows (8). Each benchmark requires reasoning about shape constraints across multiple computation steps—a capability that syntactic pattern matching fundamentally lacks.

TensorGuard achieves F1=1.00 (P=1.00, R=1.00). The syntactic baseline detects zero bugs (F1=0.00)—it cannot trace dimensions through projection layers or reason about broadcasting

Table 3: Suite B aggregate results on 205 benchmarks across 8 categories.

Tool	F1	P	R	Acc	FP	FN
TensorGuard	0.897	1.000	0.813	0.912	0	17
Syntactic	0.404	1.000	0.253	0.668	0	68
GPT-4.1-nano	0.842	0.786	0.907	0.839	21	8

compatibility. Average verification time is 261ms per benchmark (median 362ms, p95=1195ms, p99=1607ms, max=2678ms). The gap between F1=1.00 and F1=0.00 demonstrates that Z3-backed constraint propagation provides a qualitative capability advantage over syntactic analysis for architecture-level bugs.

8.2 Suite B: Expanded Evaluation (205 Benchmarks)

We evaluate all three tools on an expanded suite of 205 benchmarks spanning 8 categories drawn from realistic PyTorch code patterns.

Benchmark categories.

- **HuggingFace-style** (35): Transformer-based architectures modeled after HuggingFace patterns, including attention, MLP, and encoder-decoder compositions.
- **Vision** (31): Convolutional architectures including ResNet-style, VGG-style, and multi-branch networks with spatial dimension tracking.
- **Device** (20): Modules with device placement bugs (partial `.cuda()` calls, cross-device operations).
- **Phase** (20): Modules with train/eval phase bugs (missing `model.eval()`, inconsistent submodule phases).
- **Broadcast** (29): Operations requiring NumPy-style broadcasting reasoning across multiple dimensions.
- **Reshape** (19): Reshape, view, and permute operations with element count preservation constraints.
- **Chain** (21): Multi-step linear chains where dimension mismatches propagate across several layers.
- **Correct** (30): Bug-free modules that should be verified as SAFE (testing false positive rate).

Baselines. The *syntactic baseline* checks adjacent layer parameter compatibility (e.g., `Linear(m, n)` followed by `Linear(n, k)`) without Z3 reasoning. The *GPT-4.1-nano LLM baseline* receives each module's source code with the prompt: "Does this PyTorch nn.Module have a shape mismatch bug? Answer YES or NO, then explain."

Table 4: Per-category F1 scores on 205 benchmarks. Bold indicates the best F1 in each category.

Category	n	**TensorGuard**		**Syntactic**		**GPT-4.1-nano**	
		F1	P/R	F1	P/R	F1	P/R
HuggingFace	35	0.74	1.0/.59	0.38	1.0/.24	**0.84**	.80/.88
Vision	31	**0.90**	1.0/.87	0.35	1.0/.21	0.79	.75/.84
Device	20	**1.00**	1.0/1.0	0.00	—/0.0	0.72	.69/.75
Phase	20	**1.00**	1.0/1.0	0.00	—/0.0	0.90	.82/1.0
Broadcast	29	**0.81**	1.0/.69	0.42	1.0/.27	0.78	.74/.82
Reshape	19	**0.80**	1.0/.80	0.50	1.0/.33	0.80	.77/.84
Chain	21	**0.91**	1.0/.91	0.55	1.0/.38	0.88	.83/.93
Correct	30	—	1.0/—	—	1.0/—	—	.80/—

8.2.1 Aggregate Results

Table 3 shows the aggregate results. TensorGuard achieves F1=0.897 with P=1.000 and **zero false positives** on 205 benchmarks, *surpassing* the GPT-4.1-nano LLM baseline (F1=0.842) by 5.5 F1 points. The syntactic baseline achieves F1=0.404 with perfect precision but very low recall. TensorGuard achieves perfect precision (1.000 vs. 0.786) with 0 false positives versus the LLM's 21.

8.2.2 Per-Category Breakdown

Table 4 reveals important category-level differences. TensorGuard excels in categories requiring formal reasoning:

- **Phase bugs (F1=1.00):** TensorGuard achieves perfect detection with *zero false positives*, compared to the LLM's F1=0.90 (2 false positives from hallucinated phase issues). The $\mathcal{T}_{phase}$ theory provides sound guarantees that the LLM cannot match.
- **Device bugs (F1=1.00):** TensorGuard achieves perfect detection of device placement bugs, detecting plain tensor attributes, tensor factory calls without `device=` kwargs in `forward()`, and plain lists of tensors. The LLM achieves only F1=0.72.
- **Vision bugs (F1=0.90):** Improved convolution analysis with proper extraction of padding, stride, and dilation from `Conv2d` layer definitions ($H' = \lfloor (H + 2p - d(k-1) - 1)/s + 1 \rfloor$) yields P=1.00, surpassing the LLM (F1=0.79).
- **Broadcast bugs (F1=0.81):** The `BroadcastPropagator` reasons precisely about NumPy broadcasting rules, outperforming the LLM (F1=0.78).

The LLM outperforms TensorGuard only on:

- **HuggingFace (F1=0.84 vs. 0.74):** The LLM recognizes common Transformer patterns and their expected dimensions from training data. TensorGuard maintains P=1.000 in this category.

8.2.3 False Positive Analysis

TensorGuard produces **zero false positives** on Suite B (P=1.000). This perfect precision is a consequence of the conservative verification strategy: TensorGuard only reports a bug when the Z3 solver produces a concrete counterexample, never from heuristic pattern matching.

Table 5: Per-model Z3 solver statistics. All four theory plugins (BroadcastPropagator, StridePropagator, DevicePropagator, PhasePropagator) are actively exercised on every model.

Model	Steps	Z3 Queries	UNSAT	SAT	Time (ms)
MLP-2	2	45	35	10	1.4
CNN-2	2	45	35	10	1.3
Transformer-2	2	45	35	10	1.4
Deep-4	4	79	63	16	2.1
AutoEncoder	2	45	35	10	1.6
Dropout-3	3	60	45	15	1.8
BatchNorm-3	3	64	51	13	1.8
ResBlock	3	60	45	15	2.2
Wide-3	3	62	49	13	1.9
Total	—	505	393	112	15.5

The LLM produces 21 false positives, spread across all categories. The LLM's false positives are *hallucinations*: the model confidently reports bugs that do not exist, with no formal basis for the claim.

8.2.4 False Negative Analysis

TensorGuard produces 17 false negatives. The primary sources:

- **HuggingFace (unsupported patterns):** Complex cross-attention modules and dynamic dimension computation in encoder-decoder architectures.
- **Vision/reshape:** Unsupported architectural patterns including `ConvTranspose2d`, adaptive pooling, and dynamic `view` with computed dimensions.

8.3 Confidence Scoring

Each verification verdict carries a confidence level (High, Medium, or Low) based on the information available: High when all dimensions are concrete and Z3-proven; Medium when symbolic dimensions are resolved via contract discovery; Low for heuristic-based verdicts with missing models. The `filter_by_confidence` API allows users to suppress low-confidence warnings in production use.

With 205 benchmarks and P=1.000, TensorGuard has zero false positives. For recall of 0.813, the Wilson 95% confidence interval is approximately (0.721, 0.880).

8.4 Z3 Solver Statistics

8.5 Contract Discovery Ablation

We evaluate three contract discovery configurations on 8 benchmarks (4 buggy, 4 correct) across standard architecture families.

All three configurations achieve identical F1=1.00, confirming that quality-filtered contract discovery causes no degradation. This is the primary concern given that unfiltered refinement can over-constrain the input space at scale.

Table 6: Contract discovery ablation on 8 benchmarks. All configurations achieve F1=1.00, confirming quality filtering causes no degradation.

Configuration	**Precision**	**Recall**	**F1**
No refinement (single-pass)	1.00	1.00	1.00
Refinement, quality filter OFF	1.00	1.00	1.00
Refinement, quality filter ON	1.00	1.00	1.00

Table 7: Summary of TensorGuard evaluation across all three suites.

Evaluation	n	**F1**	**P**	**R**	**Key finding**
Suite A (theory)	18	1.00	1.00	1.00	Z3 theories essential
Suite B (all)	205	0.897	1.000	0.813	0 FP, leads LLM by 5.5 pts
Phase	20	1.00	1.00	1.00	0 FP, formal guarantee
Device	20	1.00	1.00	1.00	0 FP, formal guarantee
Vision	31	0.90	1.00	0.87	beats LLM
Chain	21	0.91	1.00	0.91	strong chains
Broadcast	29	0.81	1.00	0.69	beats LLM
Suite C (real-world)	56	0.925	1.000	—	competitive with LLM

8.6 Summary of Results

The key findings are:

1. TensorGuard **leads the LLM baseline by 5.5 F1 points on Suite B** (0.897 vs. 0.842) with P=1.000 and zero false positives, while providing formal guarantees that no LLM can match: SMT-LIB 2.6 proof certificates for verified-safe models.

2. On **Suite C (56 real-world benchmarks)**, TensorGuard achieves F1=0.925 (P=1.000, 0 FP), competitive with GPT-4.1-nano (F1=0.933) while maintaining perfect precision and formal guarantees.

3. **Perfect scores on Phase and Device** (F1=1.00 each) demonstrate domains where constraint-based verification is provably complete. The product theory $\mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase}$ is fully functional.

4. The **syntactic baseline** (F1=0.404) confirms that Z3-backed constraint propagation provides substantial value over pattern matching alone.

8.7 Suite C: Real-World Benchmarks (56 Benchmarks)

To evaluate generalization, we curated Suite C: 56 real-world `nn.Module` classes drawn from public GitHub repositories, including production models, tutorial code, and research prototypes.

On Suite C, GPT-4.1-nano achieves a slightly higher F1 (0.933 vs. 0.925), but TensorGuard maintains P=1.000 with zero false positives. The LLM's advantage comes from its ability to recognize common architectural patterns from training data, while TensorGuard relies solely on constraint propagation. The key differentiator is reliability: every TensorGuard verdict is backed by a Z3 proof or counterexample, whereas the LLM's verdicts are probabilistic with no formal guarantee. TensorGuard is now competitive with the LLM on real-world code while maintaining

Table 8: Suite C results on 56 real-world benchmarks.

Tool	F1	P	R	FP	FN
TensorGuard	0.925	1.000	—	0	—
GPT-4.1-nano	0.933	—	—	—	—

perfect precision—a property critical for adoption in CI/CD pipelines where false alarms cause tool abandonment [19].

9 Proof Certificates

A key differentiator between TensorGuard and LLM-based bug detectors is *verifiability*: when TensorGuard reports a model as SAFE, it emits an SMT-LIB 2.6 proof certificate that can be independently checked by any conforming SMT solver (Z3, CVC5, etc.). LLMs provide opinions; TensorGuard provides certificates.

Definition 17 (Safety Certificate)**.** *A* safety certificate *for an* `nn.Module` M *is a tuple* $\mathcal{C} = (m, \Pi, Q, \mathcal{T}, h)$ *where:*

- m *is the module name;*
- Π *is the set of properties proved (shape compatibility, device consistency, phase correctness);*
- Q *is the Z3 query statistics (number of queries, UNSAT/SAT breakdown, solving time);*
- $\mathcal{T}$ *is the theory information (*$\mathcal{T}_{shape}$*,* $\mathcal{T}_{device}$*,* $\mathcal{T}_{phase}$*, or their product);*
- h *is a SHA-256 fingerprint of the source module, binding the certificate to a specific code version.*

The certificate encodes the verification conditions as an SMT-LIB 2.6 script. Independent verification reduces to checking that the conjunction of all encoded constraints is UNSAT (i.e., no assignment violates safety). This provides a *machine-checkable proof* that the model's computation graph is free of shape, device, and phase errors—a guarantee that no LLM baseline can provide.

10 Soundness of Finite-Domain Theories

The perfect F1=1.00 scores on Phase and Device categories are not coincidental: for these finite-domain theories, the verification is provably complete.

Theorem 18 (Soundness and Completeness for Finite-Domain Theories)**.** *Let* $\mathcal{T}$ *be a theory with finite sort of cardinality* n*. For any quantifier-free formula* ϕ *over* $\mathcal{T}$*, constraint-based verification via the corresponding* `UserPropagator` *is both sound and complete.*

Proof. Soundness follows from the `UserPropagator` contract: every propagation and conflict is a logical consequence of the theory axioms applied to fixed variable assignments (Lemmata in Appendix C). Completeness follows from the finite domain: the propagator exhaustively enumerates all possible assignments (at most n^k for k variables), so no satisfying assignment is missed. For $\mathcal{T}_{phase}$ ($n = 2$) and $\mathcal{T}_{device}$ ($n \leq 5$ in practice), this enumeration is efficient. □

Corollary 19 (Product Theory Completeness for Phase–Device)**.** *The product $\mathcal{T}_{device} \times \mathcal{T}_{phase}$ is both sound and complete under Tinelli–Zarba arrangement enumeration [6], since both components have finite sorts and the arrangement enumeration is exhaustive (Definition 10).*

11 Related Work

PyTea (OOPSLA 2022). PyTea [8] performs abstract interpretation for tensor shape analysis over PyTorch programs. It reasons about individual operations and simple sequential compositions using abstract domains. TensorGuard differs in three key respects: (1) TensorGuard uses SMT with UserPropagator plugins instead of abstract domains, enabling richer constraint reasoning and theory combination; (2) TensorGuard covers device consistency, phase correctness, and broadcasting—not just shapes; (3) TensorGuard verifies the full `nn.Module` computation graph composition, not individual operations.

Pyright + tensor stubs. Pyright [9] with tensor type stubs enables type-level shape tracking through generic type parameters. This requires explicit type annotations on all tensor-producing functions and layer declarations. TensorGuard works on unannotated Python with zero developer effort: it extracts shape constraints directly from `nn.Module __init__` and `forward` methods without requiring any annotations.

mypy. mypy [10] provides general-purpose static type checking for Python. It verifies surface-level types (e.g., `Tensor` vs. `int`) but has no tensor shape reasoning at all: it cannot detect that `nn.Linear(512, 256)` followed by `nn.Linear(128, 64)` is a shape mismatch.

Shape guards in JAX and TorchScript. JAX's shape checking and TorchScript [11] compilation use runtime tracing to discover tensor shapes. These are dynamic approaches that require executing the model with concrete inputs. TensorGuard performs fully static verification without executing any code, catching bugs before training begins. jaxtyping [12], beartype [13], and torchtyping [14] provide runtime shape assertions but no static guarantees.

Houdini-style predicate discovery. Houdini [1] (FME 2001) discovers annotation candidates by eliminating non-inductive predicates from a large candidate set. TensorGuard's contract discovery loop is a Houdini-style predicate accumulation loop that *accumulates* predicates monotonically from counterexamples, substantially simpler than general abstraction refinement [4]: deterministic refinement via layer parameters and unsat core-based predicate extraction.

Domain-specific SMT theories. Custom theory plugins via Z3's `UserPropagator` API [18] enable domain-specific decision procedures in the DPLL(T) loop. TensorGuard contributes the first theories for tensor operations.

Theory combination. Nelson–Oppen [5] combination requires stably infinite theories. Tinelli–Zarba [6] extend this to theories with finite domains via arrangement enumeration. TensorGuard's theory combination applies the Tinelli–Zarba method to combine $\mathcal{T}_{shape}$ (stably infinite) with $\mathcal{T}_{device}$ and $\mathcal{T}_{phase}$ (finite domains).

Abstract interpretation. TensorGuard's approach—forward symbolic propagation of constraints through a data-flow graph—is related to abstract interpretation [20] with domain-specific abstract domains. The product theory can be viewed as a reduced product of three abstract domains, with Z3 providing the concretization. The key difference is that our abstract domains are implemented as SMT theories with eager propagation via UserPropagator plugins, enabling fine-grained interaction with Z3's DPLL(T) search.

Behavioral neural network verification. Reluplex, ERAN, and α-β-CROWN verify *behavioral* properties (robustness, reachability). TensorGuard performs *computation graph verification*: structural correctness of shape compatibility, device consistency, and phase correctness. These are complementary.

ML-specific dynamic analysis. CrossHair [15] performs symbolic execution of Python but does not target tensor shapes. TensorFuzz [16] applies fuzz testing. DeepLocalize [17] localizes faults dynamically.

12 Discussion and Limitations

Positioning. TensorGuard is a guard-harvesting constraint verification tool with domain-specific SMT theories for tensor operations. It performs Houdini-style predicate accumulation with Z3 UserPropagator product theory, closer to abstract interpretation with Z3-backed abstract domains than to classical model checking or behavioral neural network verification. The computation graphs we analyze are acyclic DAGs, not Kripke structures; our contract discovery uses Z3-backed abstract feasibility checking (not concrete Python execution); and we verify structural properties, not behavioral ones.

Device bug detection: from limitation to strength. TensorGuard now achieves perfect F1=1.00 on the device category (20 benchmarks). The `DevicePropagator` was extended to detect three patterns: (a) plain tensor attributes (e.g., `self.buffer = torch.zeros(...)`) lacking explicit device placement; (b) tensor factory calls (`torch.zeros`, `torch.ones`, `torch.randn`) in `forward()` without a `device=` keyword argument; (c) plain Python lists of tensors that bypass `nn.ParameterList` / `nn.ModuleList` device propagation. The $\mathcal{T}_{device}$ theory detects device inconsistencies when device assignments are statically determinable, and the extended pattern checker covers the common cases that previously required runtime information.

Precision. TensorGuard achieves P=1.000 on both Suite B and Suite C with zero false positives. This is a consequence of properly extracting padding, stride, and dilation from `Conv2d` layer definitions, using the full output formula $H' = \lfloor (H + 2p - d(k-1) - 1)/s + 1 \rfloor$, and the conservative verification strategy that only reports bugs backed by Z3 counterexamples.

Positioning relative to the LLM baseline. TensorGuard leads GPT-4.1-nano by 5.5 F1 points on Suite B (0.897 vs. 0.842) and is competitive on Suite C (0.925 vs. 0.933). Beyond the F1 comparison, the tools have fundamentally different reliability profiles:

- TensorGuard provides *formal guarantees*: when it reports SAFE for a phase, device, or chain property, the guarantee is backed by a Z3 proof. For verified-safe models, TensorGuard emits SMT-LIB 2.6 proof certificates independently checkable by Z3, CVC5, or any conforming solver (§9). The LLM provides no such guarantee.

- TensorGuard achieves P=1.000 with zero false positives on both Suite B and Suite C. The LLM's false positives are *hallucinations* (confidently wrong with no formal basis).
- In safety-critical deployment, formal guarantees with known limitations are preferable to higher-recall heuristics with unpredictable failure modes.

The combination of competitive F1, perfect precision, and formal certificates makes TensorGuard a practical verification tool for CI/CD integration.

False negative analysis. TensorGuard misses bugs in HuggingFace, vision, and reshape categories. These stem from the AST-based graph extractor's inability to track shapes through: (1) complex multi-hop attribute access (e.g., `self.model.encoder`); (2) dynamic reshape with fully symbolic dimensions; (3) loop-based layer application with data-dependent indexing. These are engineering limitations, not fundamental to the approach. The graph extractor now supports `ConvTranspose2d`, `F.interpolate`, element-wise multiply (`x * y`), and `view(x.size(0), -1)` flattening patterns, which previously caused false positives on architectures like U-Net, SE-Net, ResNet skip connections, and BERT.

Comparison with PyTea. PyTea [8] performs abstract interpretation over individual tensor operations and simple sequential compositions. TensorGuard differs in three ways: (1) it verifies *composition* of layers across the full computation graph, not just individual operations; (2) it uses SMT with Z3 `UserPropagator` plugins instead of abstract domains, covering device, phase, and broadcast—not just shapes; (3) it discovers implicit shape contracts via Houdini-style predicate accumulation with Z3 UserPropagator product theory.

Other limitations. The current AST-based extraction cannot always resolve variable constructor parameters beyond simple defaults. Skip connections with channel changes can produce conservative results when concatenation tracking is incomplete. Recurrent architectures are not handled by the acyclic assumption.

Future work. Adding dtype theory and gradient flow theory. Scaling to 100+ layer models via incremental verification. Integration with `torch.compile`'s symbolic shape infrastructure (TorchDynamo shape guards, `torch.export` Dim constraints).

A Decidability Proof for $\mathcal{T}_\Pi$

Proof. We show decidability using the Tinelli–Zarba extension [6].

1. Sort structure. $\mathcal{T}_{shape}$ is stably infinite (models extend with fresh tuples). $\mathcal{T}_{device}$ has finite sort ($|Dev| = n$). $\mathcal{T}_{phase}$ has $|Phase| = 2$. The Tinelli–Zarba extension handles finite sorts by enumerating arrangements.

2. Signature disjointness. $\mathcal{T}_{shape}$ uses $\{get, len, set, prod\}$. $\mathcal{T}_{device}$ uses $\{dev\}$. $\mathcal{T}_{phase}$ uses $\{phase\}$. Cross-sort axioms use only shared equality variables.

3. Component decidability. $\mathcal{T}_{shape}$: QF-LIA decidable [7]; QF-NIA decidable with bounded variables. $\mathcal{T}_{device}$: finite domain enumeration. $\mathcal{T}_{phase}$: two-element algebra.

4. Combination. For k shared variables over a finite sort of cardinality n, the Tinelli–Zarba method enumerates all partitions of the k variables into at most n equivalence classes (arrangements). For each arrangement δ, each component solver checks satisfiability of its purified formula conjoined with δ. If all solvers agree on some arrangement, the combined formula is satisfiable. The number of arrangements is bounded by $\sum_{j=1}^{\min(k,n)} S(k,j)$ where $S(k,j)$ is the Stirling number of the second kind. For typical computation graphs ($k \leq 3$, $n = 5$), this is at most $S(3,1) + S(3,2) + S(3,3) = 1 + 3 + 1 = 5$ arrangements for device variables. □

Proposition 20 (Combined Complexity)**.** *(1) PTIME without reshape. (2) NP with reshape (bounded variables). (3) NP-hard for symbolic reshape (reduction from PARTITION).*

Theorem 21 (NP-Completeness of Reshape Satisfiability)**.** *The satisfiability problem for reshape constraints with symbolic dimensions is NP-complete.*

Proof. **NP membership.** Given a candidate assignment of dimension values, verifying that all reshape constraints $\prod s_{in}[i] = \prod s_{out}[j]$ are satisfied is polynomial (multiply and compare).

NP-hardness (reduction from PARTITION). Given a PARTITION instance $S = \{s_1, \ldots, s_k\}$ with $\sum_{i=1}^{k} s_i = 2T$, we construct a reshape constraint as follows. Define a flat tensor of $\prod_{i=1}^{k} s_i$ elements (one dimension per element of S, with size s_i along axis i). The reshape target is shape $(T,\ \prod_{i=1}^{k} s_i/T)$.

This reshape constraint is satisfiable if and only if $T \mid \prod_{i=1}^{k} s_i$, which requires selecting a subset $S' \subseteq S$ whose product equals T. But since the s_i are given as a PARTITION instance, the existence of such a subset S' with $\prod_{i \in S'} s_i = T$ is equivalent to finding a subset with $\sum_{i \in S'} s_i = T$ when each s_i is encoded in unary (the standard PARTITION reduction uses additive structure).

More precisely: construct dimension variables $d_1, \ldots, d_k$ with $d_i \in \{1, s_i\}$ (binary choice per element) and assert the reshape constraint $\prod_{i=1}^{k} d_i = T$. Each $d_i = s_i$ corresponds to including s_i in the subset; $d_i = 1$ corresponds to excluding it. The constraint $\prod d_i = T$ is satisfiable iff PARTITION has an equal-product subset, which encodes the PARTITION problem. □

B Soundness Proof

Theorem 22 (Verification Soundness)**.** *If* TensorGuard *reports* Safe*, then for all concrete inputs x satisfying the discovered contracts $\mathcal{P}$, the* `nn.Module`*'s* `forward()` *method does not raise a shape, device, or phase error at any computation step.*

Proof. By induction on the structure of the computation graph $G = (V, E, \lambda, \sigma_0)$.

Base case. At the input node $v_0 \in V_{in}$, the initial state $\sigma_0 = (s_0, d_0, p_0)$ satisfies all discovered contracts $\mathcal{P}$ by assumption.

Inductive step. Assume all predecessors of node v satisfy the safety property φ. For edge $(u, v) \in E$ with layer $\ell = \lambda(u, v)$:

1. **Shape:** The shape transfer function $\tau_\ell^s : \mathbb{Z}^* \to \mathbb{Z}^*$ is sound by construction: it overapproximates the actual PyTorch semantics. For `nn.Linear(m,n)`, the precondition $last_dim(s_u) = m$ and postcondition $s_v = s_u[-1{:=}n]$ exactly match PyTorch's behavior. For broadcasting (Definition 12), the `BroadcastPropagator` implements the NumPy specification. The verifier encodes these constraints in $\mathcal{T}_{shape}$ and Z3 checks satisfiability. Since Z3 is a sound solver, UNSAT implies no concrete input violates the shape precondition.

2. **Device:** The `DevicePropagator` enforces $\mathcal{T}_{device}$ axioms via eager propagation. It raises conflicts only on genuine device mismatches (Appendix C). When the propagator reports no conflict, devices are consistent.

3. **Phase:** The `PhasePropagator` enforces $\mathcal{T}_{phase}$ axioms. When it deduces $dropout_active(m) = false$ from $phase(m) = \texttt{eval}$, this matches PyTorch's `nn.Module.eval()` semantics.

4. **Theory combination:** The three theories are combined via Tinelli–Zarba arrangement enumeration (Algorithm 1). The signatures are disjoint ($\Sigma_{shape} \cap \Sigma_{device} \cap \Sigma_{phase} = \emptyset$), and cross-sort constraints use only shared equality variables. Soundness of the combination follows from Theorem 11.

Since the graph G is acyclic (a DAG), the induction is well-founded and terminates.

Limitations. Unsupported operations may cause TensorGuard to miss bugs (false negatives). The soundness guarantee applies only to computation steps whose shape transfer functions are modeled. The graph extractor now covers `nn.ConvTranspose2d`, `F.interpolate`, element-wise multiply/subtract, `nn.ModuleList`, and common `view(x.size(0), -1)` flattening patterns.

Mechanized proofs. The core soundness theorems are mechanized in Lean 4 (`lean/TheoryCombination.lean`), including: theory combination soundness (Theorem 11), broadcast commutativity and associativity, non-tautological propagator soundness (connecting the executable boolean checker to the semantic specification via `broadcastDim_sound`), CEGAR convergence from finite predicate universe, and the forward direction of the PARTITION reduction for NP-completeness. □

C UserPropagator Soundness

Each `UserPropagator` plugin satisfies a soundness invariant: every propagation and conflict is a logical consequence of the theory axioms applied to fixed variable assignments.

Lemma 23 (BroadcastPropagator Soundness)**.** *Every propagation $d_i^{out} = v$ by the `BroadcastPropagator` follows from axioms A1–A2 (Appendix E) applied to concrete values of d_i^A and d_i^B. Every conflict clause $\neg(d_i^A{=}a \wedge d_i^B{=}b)$ follows from axiom A3.*

Proof. The propagator only fires in the `fixed` callback when both d_i^A and d_i^B have concrete values. It checks the three cases of A1 (equal, left-broadcast, right-broadcast) and propagates the unique result from A2. When none of the three cases applies, $a \neq b$, $a \neq 1$, $b \neq 1$, so A3 yields the conflict. No speculative deductions are made on unassigned variables. □

Lemma 24 (StridePropagator Soundness)**.** *The `StridePropagator` computes stride values purely from fixed dimension values using the contiguous stride formula. Conflicts are raised only when computed values are inconsistent with declared values.*

Lemma 25 (DevicePropagator Soundness)**.** *The `DevicePropagator` propagates device equalities from the device axioms of $\mathcal{T}_{device}$ and raises conflicts only on device inequality between operands of binary operations or between layer parameters and input tensors.*

Lemma 26 (PhasePropagator Soundness)**.** *The `PhasePropagator` deduces phase-dependent behavior only from fixed phase assignments, following the axioms of $\mathcal{T}_{phase}$.*

D Shape Algebra

D.1 Broadcasting

NumPy/PyTorch broadcasting aligns shapes from the right. Given $A : (s_1, \ldots, s_m)$ and $B : (t_1, \ldots, t_n)$, broadcasting succeeds iff for each aligned pair: $s_i = t_j$, $s_i = 1$, or $t_j = 1$. Z3 encoding: $\bigwedge_{k=1}^{\max(m,n)} (s'_k = t'_k \vee s'_k = 1 \vee t'_k = 1)$ where s', t' are 1-padded.

D.2 Matrix Multiplication

`A @ B`: $cols(A) = rows(B)$, result $(\ldots, m, n)$.

D.3 Reshape

$\prod s_x[i] = \prod s[j]$, using QF-NIA for symbolic dimensions.

E Formal Specification of $\mathcal{T}_{broadcast}$

We give a self-contained axiomatization of the broadcast theory implemented by the `BroadcastPropagator`.

Signature.

- **Sorts:** *Dim* (integers ≥ 1), *Shape* (finite tuples over *Dim*), *Idx* (natural numbers).
- **Functions:** $get : Shape \times Idx \to Dim$, $len : Shape \to Idx$, $bc : Shape \times Shape \to Shape$ (broadcast result), $pad : Shape \times Idx \to Shape$ (1-pad to target length).
- **Predicates:** $bcompat : Dim \times Dim$ (dimension-level broadcast compatibility), $scompat : Shape \times Shape$ (shape-level broadcast compatibility), $mcompat : Shape \times Shape$ (matmul compatibility).

Axioms.

A1 (Compatibility) $bcompat(a, b) \;\Leftrightarrow\; a = b \;\vee\; a = 1 \;\vee\; b = 1$.

A2 (Broadcast result) $bcompat(a, b) \;\Rightarrow\; bc_dim(a, b) = \max(a, b)$.

A3 (Incompatibility) $\neg bcompat(a, b) \;\Rightarrow\; bc(s_A, s_B)$ is undefined (conflict).

A4 (Shape compatibility) $scompat(s_A, s_B) \;\Leftrightarrow\; \forall i < \max(len(s_A), len(s_B)).\ bcompat(get(pad(s_A), i),\ get(pa$ where shapes are right-aligned via 1-padding.

A5 (Matmul compatibility) $mcompat(s_A, s_B) \;\Leftrightarrow\; get(s_A, len(s_A)-1) = get(s_B, len(s_B)-2)$.

A6 (Positive dimensions) $\forall s.\ \forall i < len(s).\ get(s, i) \geq 1$.

Decision procedure. For concrete ranks (known $len(s_A)$ and $len(s_B)$), the universal quantifier in A4 unfolds to a finite conjunction and the theory reduces to QF-LIA, which is decidable in polynomial time. The `BroadcastPropagator` implements this reduction eagerly: when Z3 fixes dimension values, the propagator checks A1 for each aligned pair and propagates A2 or raises a conflict via A3.

Soundness. The propagator only deduces consequences from fixed variable assignments: it never speculates on unassigned dimensions. Every propagation ($d_i^{out} = v$) follows from A1–A2 applied to concrete values of d_i^A and d_i^B. Every conflict clause ($\neg(d_i^A{=}a \wedge d_i^B{=}b)$) follows from A3 applied to values where $a \neq b$, $a \neq 1$, $b \neq 1$. Thus the propagator is a sound theory solver for $\mathcal{T}_{broadcast}$.

Theory combination. The device and phase theories have finite sorts. Combination with $\mathcal{T}_{broadcast}$ follows the Tinelli–Zarba method [6] for combining theories where some component theories have finite domains: purify shared terms, enumerate finite-sort arrangements, and check each component theory independently.

F Formal Specification of $\mathcal{T}_{stride}$

We give a self-contained axiomatization of the stride theory implemented by the `StridePropagator`.

Signature. $\Sigma_{stride} = (S, F, P)$ where:

- **Sorts:** *Dim* (positive integers ≥ 1), *Stride* (positive integers ≥ 1).
- **Functions:** $cstride : Dim^n \to Stride^n$ (contiguous stride computation), $numel : Dim^n \to \mathbb{Z}$ (total element count $= \prod shape[i]$).
- **Predicates:** $contiguous : Dim^n \times Stride^n$ (stride matches contiguous layout), $reshape_ok : Dim^m \times Dim^n$ (element count preservation for reshape).

Axioms.

A1 (Contiguous definition) $contiguous(s, t) \;\Leftrightarrow\; t[n{-}1] = 1 \;\wedge\; \forall i < n{-}1.\; t[i] = t[i{+}1] \times s[i{+}1]$.

A2 (Reshape validity) $reshape_ok(s_{in}, s_{out}) \;\Leftrightarrow\; numel(s_{in}) = numel(s_{out})$, i.e., $\prod_i s_{in}[i] = \prod_j s_{out}[j]$.

A3 (Convolution output) For Conv2d with kernel k, stride s, padding p, dilation d: $H' = \lfloor (H + 2p - d(k-1) - 1)/s \rfloor + 1$.

A4 (Positive dimensions) $\forall i.\; s[i] \geq 1 \;\wedge\; t[i] \geq 1$.

Decision procedure. Contiguous stride constraints (A1) are linear (QF-LIA) for concrete shapes. Reshape constraints (A2) involve multiplication of variables (QF-NIA), handled by Z3's nonlinear solver. The `StridePropagator` eagerly propagates stride values when shape dimensions are fixed: given concrete values $s[0], \ldots, s[n{-}1]$, it immediately computes $t[i] = \prod_{j>i} s[j]$ and propagates these equalities.

Soundness. The propagator computes strides purely from fixed dimension values using the formula in A1. Conflicts are raised only when the computed strides are inconsistent with user-declared strides (e.g., a non-contiguous tensor used with an operation requiring contiguity).

G Full Benchmark Descriptions

G.1 Suite A: Theory-Exercising (18 benchmarks)

Broadcasting (6). Compatible/incompatible additions, batched matmul with broadcast, multi-step broadcasting pipeline, symbolic broadcast, residual broadcast.

Matrix multiplication (4). Chain, mismatch, transpose, batched matmul with dimension flow.

Multi-step flows (8). Linear chain, conv+pool+flatten+linear, skip connection, attention flow, encoder-decoder, reshape between layers, concatenation tracking, 6-layer mixed.

G.2 Suite B: Expanded Evaluation (205 benchmarks)

HuggingFace-style (35). Transformer encoders, BERT-style attention, GPT blocks, encoder-decoder models, multi-head attention variants, feed-forward networks with dimension mismatches, layer normalization chains, and cross-attention modules.

Vision (31). ResNet-style residual blocks, VGG-style sequential convolutions, multi-branch networks (Inception-style), feature pyramid networks, U-Net encoder-decoder paths, DCGAN generators, squeeze-and-excitation blocks, and depthwise separable convolutions.

Device (20). Partial `.cuda()` placement, cross-device binary operations, conditional device placement (`if cuda_available`), mixed-device sequential modules, and data-parallel edge cases.

Phase (20). Missing `model.eval()`, inconsistent submodule phases, dropout active during inference, batch normalization with single-element batches in eval mode, and phase-dependent branching with incorrect guard conditions.

Broadcast (29). Compatible and incompatible element-wise operations across varying ranks, batched broadcasting with symbolic dimensions, multi-step broadcasting pipelines, broadcast with reshape interactions, and attention score broadcasting patterns.

Reshape (19). Element count mismatch in `view`, `reshape` with `-1` inference, transpose followed by contiguous reshape, flatten-to-linear dimension chains, and permute/reshape composition errors.

Chain (21). Multi-layer linear chains (3–8 layers) with dimension mismatches at various depths, chains with interleaved normalization and activation, bottleneck patterns (narrow-wide-narrow), and autoencoder chains with symmetric dimension requirements.

Correct programs (30). Bug-free implementations of standard architectures including MLPs, CNNs, Transformers, autoencoders, GANs, attention mechanisms, residual networks, and mixed architectures. These test the false positive rate.

G.3 Suite C: Real-World Benchmarks (56 benchmarks)

56 `nn.Module` classes drawn from public GitHub repositories, covering production models (torchvision, HuggingFace Transformers), tutorial code (PyTorch official tutorials), and research prototypes (papers with code). Includes standard architectures (ResNet, BERT, GPT-2, U-Net, DCGAN) and domain-specific models (medical imaging, point cloud processing, audio classification).

G.4 Contract Discovery Ablation (8 benchmarks)

4 buggy, 4 correct across MLP, CNN, Transformer, and ResNet families.

H Contract Discovery Algorithm

Algorithm 4 ExtractFromUnsatCore: extract a shape predicate from a spurious counterexample via Z3 unsat core.

Require: Counterexample cex, computation graph G
Ensure: New predicate p

```
edge ← cex.failing_edge
ℓ ← λ(edge)
φ_neg ← ¬φ(σ_edge)
core ← Z3.unsat_core(φ_neg ∧ ⋀P)
p ← ProjectToPredicateLanguage(core)
if edge.target has successor edge e′ then
    ℓ′ ← λ(e′)
    c ← in_dim(ℓ′)
    return last_dim(s_out) = c
else
    s_in ← cex.source_state.shape
    s_out ← τ_ℓ(s_in)
    return shape(edge.target) = s_out
end if
```

This uses *unsat core-based predicate extraction*. Unsat cores lack the vocabulary restriction and entailment properties of interpolants. The projection is sound but not complete.

I Guard Harvesting

TensorGuard harvests predicates from seven syntactic sources: conditional guards, early-exit guards, assertions, default values, exception handlers, walrus operators, and comprehension filters. Guard harvesting achieves P=1.00, R=0.90, F_1=0.95 on null-safety benchmarks.

Table 9: Guard qualifier transfers.

Guard pattern	Γ^+	Γ^-
`x is None`	$v = \texttt{None}$	$v \neq \texttt{None}$
`x is not None`	$v \neq \texttt{None}$	$v = \texttt{None}$
`isinstance(x,T)`	$v \neq \texttt{None}$, $\mathit{type}(v){=}T$	$\mathit{type}(v) \neq T$
`x == c`	$v = c$	$v \neq c$
`x > c`	$v > c$	$v \leq c$
`hasattr(x,a)`	$\mathit{hasattr}(v, a)$, $v \neq \texttt{None}$	—

References

[1] C. Flanagan and K. R. M. Leino. Houdini, an annotation assistant for ESC/Java. In *FME*, volume 2021 of *LNCS*, pages 500–517. Springer, 2001.

[2] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided abstraction refinement. In *CAV*, pages 154–169. Springer, 2000.

[3] G. Nelson and D. C. Oppen. Simplification by cooperating decision procedures. *ACM TOPLAS*, 1(2):245–257, 1979.

[4] C. Tinelli and M. T. Zarba. Combining nonstably infinite theories. *Journal of Automated Reasoning*, 34(3):209–238, 2005.

[5] M. Presburger. Über die Vollständigkeit eines gewissen Systems der Arithmetik ganzer Zahlen, in welchem die Addition als einzige Operation hervortritt. In *Comptes Rendus du I Congrès de Mathématiciens des Pays Slaves*, pages 92–101, 1929.

[6] S. Park, Y. Kim, S. Yoo, and M. Kwon. PyTea: Static analysis of PyTorch programs for tensor shape errors. In *OOPSLA*, 2022.

[7] Microsoft. Pyright: Static type checker for Python. `https://github.com/microsoft/pyright`, 2019–2024.

[8] J. Lehtosalo et al. Mypy: Optional static typing for Python. `https://mypy-lang.org/`, 2012–2024.

[9] PyTorch Team. TorchScript. `https://pytorch.org/docs/stable/jit.html`, 2018–2024.

[10] P. Kidger. jaxtyping: Type annotations for JAX, PyTorch, and NumPy. `https://github.com/patrick-kidger/jaxtyping`, 2022–2024.

[11] C. Curry. beartype: Unbearably fast runtime type checking. `https://github.com/beartype/beartype`, 2020–2024.

[12] P. Kidger. torchtyping: Runtime type annotations for PyTorch tensors. `https://github.com/patrick-kidger/torchtyping`, 2021–2023.

[13] P. Schanely. CrossHair: Symbolic execution for Python. `https://github.com/pschanely/CrossHair`, 2020–2024.

[14] A. Odena, C. Olsson, D. Andersen, and I. Goodfellow. TensorFuzz: Debugging neural networks with coverage-guided fuzzing. In *ICML*, pages 4901–4911. PMLR, 2019.

[15] M. Wardat, W. Le, and H. Rajan. DeepLocalize: Fault localization for deep neural networks. In *ICSE*, pages 251–262. IEEE, 2021.

[16] N. Bjørner, L. de Moura, and N. Nachmanson. User propagators for custom theories in Z3. Z3 documentation, `https://z3prover.github.io/api/html/classz3_1_1UserPropagateBase.html`, 2020.

[17] A. Bessey, K. Block, B. Chelf, A. Chou, B. Fulton, S. Hallem, C. Henri-Gros, A. Kamsky, S. McPeak, and D. Engler. A few billion lines of code later: Using static analysis to find bugs in the real world. *Communications of the ACM*, 53(2):66–75, 2010.

[18] P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, pages 238–252. ACM, 1977.

[19] M. Sheeran, S. Singh, and G. Stålmarck. Checking safety properties using induction and a SAT-solver. In *FMCAD*, pages 127–144. Springer, 2000.

[20] A. Biere, A. Cimatti, E. M. Clarke, and Y. Zhu. Symbolic model checking without BDDs. In *TACAS*, pages 193–207. Springer, 1999.